

DEEPDRAC: Disposition Recommendation for Alert Clusters Based on Security Event Patterns

Yang Liu^{ID}, *Member, IEEE*, Gaofei Ruan^{ID}, Zian Luo^{ID}, Shilong Zhang, Donghao Liu, Xin Fan, Yadong Zhou^{ID}, *Member, IEEE*, and Ting Liu^{ID}, *Member, IEEE*

Abstract—In the security operation center, false positive alerts generated by security devices overwhelm security operators, leading to alert fatigue and inefficiency in identifying real threats. This paper introduces DEEPDRAC, a disposition recommendation method for alert clusters that is based on security event patterns. Our main idea is to reconstruct isolated alerts into security events and capture their essential threat characteristics as patterns. By recommending pattern information, we enable batch interpretable disposal of alerts. First, DEEPDRAC aggregates correlated alerts to a graph, representing a security event. Then, it extracts the features of the security event from two aspects: basic features via statistical methods and detailed features via a carefully designed Graph Neural Network (GNN) that focuses on edge features. Since many false alerts triggered by the same cause often recur in a fixed pattern, DEEPDRAC translates basic features into interpretable descriptors to define the basic pattern, whereas GNN embeddings complement detailed semantic information, serving as the detailed pattern, together forming the pattern of the security event. The pattern describes the critical information of the security event, so security events with the same pattern are clustered for batch processing. Finally, with few manually labeled security events, DEEPDRAC can conduct automatic disposition recommendations for newly arrived alerts, significantly reducing the workload of alert analysis. We evaluate our approach on two benchmark datasets (i.e., DARPA 1999 and CIC-IDS2017) and a real-world dataset from a large power company. The extensive experimental results demonstrate that our approach can alleviate alert fatigue more efficiently and accurately than the two state-of-the-art defense approaches can.

Index Terms—False positive alerts, alert fatigue, pattern recognition, graph neural networks, semi-supervised learning.

I. INTRODUCTION

WITH the advancement of information technology, enterprise networks are facing increasingly complex cyberattacks. To safeguard network security, operators in

the Security Operations Center (SOC) analyze alerts from Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) before taking further security actions [1]. In this process, operators must use their knowledge of cyberattacks and normal traffic to determine whether an alert represents a real threat manually. However, a serious issue of alert fatigue has arisen [2], [3], [4], where operators are overwhelmed by a large number of false positives, making it difficult to detect and respond to real attacks promptly [5], [6]. A 2019 Cisco survey revealed that 41% of the 3,540 organizations surveyed receive more than 10,000 alerts per day. Among these alerts, only 50.7% were investigated and analyzed owing to the limited disposal capabilities of the SOC, and only 24.1% of the investigated alerts were considered real attacks [7]. These observations highlight the pressing need for efficient, context-aware alert disposition methods to help SOC operators identify real threats from massive alert volumes with minimal manual effort.

To address the issue of alert fatigue, researchers have conducted numerous approaches, which can be categorized into two main types. The first type enhances security devices to improve the quality of alerts while reducing the total number of alerts [5], [8], [9], [10], [11]. However, updating various types of legacy security devices in an organization has become challenging [12]. The second type designs automated alert validation solutions to alleviate the burden on operators [13], which is more practical for alleviating alert fatigue. The alert validation methods can be categorized into three types based on how they capture alert relationships: spatial-based, temporal-based, and spatiotemporal-based methods. Spatial methods [14], [15] aggregate alerts with similar fundamental attributes, such as IP addresses or host information, into higher-level meta-alerts. Temporal methods [12], [16] generate event sequences by aggregating alerts that occur in close temporal proximity, based on the premise that alerts occurring nearly simultaneously are likely to be related. Spatiotemporal methods [2], [17], [18], [19], [20], [21] combine spatial and temporal correlations to integrate alerts, preserving more complete contextual information than the previous two methods do. Modeling alerts as graphs is a representative spatiotemporal method, where the IP addresses of the alert are usually used as nodes, and an alert exists between two nodes to generate edges. Following the correlation of alerts, alert validation methods typically employ representation and classification techniques to guarantee that critical alerts receive immediate attention and response from operators. Unfortunately, existing alert validation methods still have several limitations.

Received 19 January 2025; revised 10 June 2025; accepted 12 June 2025. Date of publication 16 June 2025; date of current version 1 July 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2703500; in part by the National Natural Science Foundation of China under Grant 62293500, Grant 62293501, Grant 62293502, and Grant 62272373; and in part by the Fundamental Research Funds for the Central Universities. The associate editor coordinating the review of this article and approving it for publication was Prof. Guan Gui. (Yang Liu and Gaofei Ruan contributed equally to this work.) (Corresponding author: Ting Liu.)

Yang Liu, Gaofei Ruan, Zian Luo, Shilong Zhang, Yadong Zhou, and Ting Liu are with the MOE KLINNS Laboratory, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: tingliu@mail.xjtu.edu.cn).

Donghao Liu and Xin Fan are with State Grid Information and Telecommunication Company, Shaanxi Electric Power Corporation Ltd., Xi'an, Shaanxi 710048, China.

Data is available on-line at <http://github.com/Gaofei-Ruan/DEEPDRAC-Appendix>

Digital Object Identifier 10.1109/TIFS.2025.3580337

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

First, most alert validation methods face significant challenges in accurately reconstructing security events from raw alerts. Spatial or temporal approaches often rely on one-sided relationships, limiting their ability to capture the full event context. While spatiotemporal methods combine spatial and temporal features into graphs, they usually merge multiple distinct events into a single graph. Spatiotemporal methods typically adopt node-level, edge-level, or graph-level strategies: the node-level and edge-level methods directly perform node classification or link prediction without considering event semantics [2], [21]. On the other hand, graph-level methods decouple events through graph partitioning, but they are typically based solely on topology. This approach fails to fully exploit the semantic and contextual attributes of alerts, which leads to incomplete event reconstruction [20].

Second, most alert validation methods rely heavily on labeled samples. An increasing number of alert validation methods leverage deep learning techniques for their effectiveness in representing and classifying correlated alerts [12], [18], [20], [22], [23]. These methods typically require many labeled samples for model training, which is particularly challenging in the security domain. As a result, the dependence on labeled samples severely limits the capabilities and practical applicability of these methods.

Third, most alert validation methods lack interpretability, which limits their practical application [2], [12], [18], [20]. Deep learning approaches for alert representation and classification often produce outputs, such as predicted categories or anomaly scores, without providing explanations. As a result, operators cannot understand the rationale behind the model's decisions. This lack of interpretability undermines trust in the system and hinders its usability in critical operational environments.

In this paper, we present a semisupervised alert validation method, DEEPDRAC (**DEEP** learning **D**isposition **R**ecommendation for **A**lert **C**lusters), which recommends risk levels and disposition information for alerts based on the patterns, aiming to alleviate alert fatigue. First, to recover contextual information on security events, we model alerts within a time window as an alert graph. Second, we apply a carefully designed community detection algorithm to the alert graph to obtain subgraphs (i.e., communities of related alerts). These subgraphs preserve contextual information on security events, including spatial correlations (such as relationships between IP addresses and ports) and temporal correlations (since alerts occur within the same time window). We treat these subgraphs as security events for further analysis. Third, we extract features of the security event, which consists of both basic features via statistical methods and detailed features from Graph Neural Networks (GNNs). Then, we translate the event features into patterns, and the security events with the same patterns are clustered together. Finally, we recommend patterns for security events and automate the filtering of false-positive alerts via a semisupervised approach. With DEEPDRAC, operators can efficiently dispose of all alerts by analyzing a small subset of security events for each pattern, significantly reducing their workload. In summary, the contributions of this paper are as follows:

- A novel semisupervised deep learning-based alert validation method called DEEPDRAC is proposed, which can correlate isolated alerts into security events and describe the critical information that triggers the events, called the pattern. DEEPDRAC processes alerts in an interpretable manner based on recommendations, providing specific event descriptions while filtering false-positive alerts.
- When training the GNN module of DEEPDRAC, we design a training framework consisting of self-supervised pretraining and supervised fine-tuning, which greatly reduces the dependence on ground truth labels.
- We evaluate DEEPDRAC on two public datasets (DARPA 1999 and CIC-IDS2017) as well as a real-world dataset collected from a large power grid company, and show that DEEPDRAC outperforms state-of-the-art alert reduction methods.

The rest of the paper is organized as follows. Section II explains the motivation of DEEPDRAC through an example. Section III provides a detailed description of the three components of DEEPDRAC. Section IV shows the performance of DEEPDRAC on two public datasets and a real-world dataset, comparing it with that of state-of-the-art alert reduction methods. Section V discusses additional aspects of DEEPDRAC, such as its time efficiency and robustness. Finally, Section VI presents the conclusions.

II. MOTIVATION

We explain the motivation for using pattern recommendation methods to dispose of alerts via an example.

In real-world scenarios, many alerts are false-positives triggered by normal business activities. For example, routine asset scans conducted by IT operations teams are often misclassified as port-scanning attacks; simulated brute-force attempts during password strength testing are flagged as real credential attacks; and cross-regional traffic between branch offices and headquarters, owing to its nonstandard network characteristics, is frequently misdetected as anomalous traffic. Although these alerts may seem isolated at first glance, multiple alerts often belong to a single security event, such as “asset scanning events,” “password testing events” or “cross-regional access events.” Additionally, many security events triggered by the same business frequently occur in a fixed pattern. Security events with the same pattern typically differ only in certain noncritical features, such as whether IT operations teams scanned two assets versus four assets.

Fig. 1 illustrates an example of correlated alerts in a real enterprise intranet. The example includes four security events, where event 1 and event 3 share the same pattern. This pattern similarity indicates that these events, despite occurring in different subnets, can be handled via a unified disposition strategy. The ability to identify and group such events is critical for reducing alert fatigue and improving operational efficiency. However, accurately reconstructing security events from a vast number of raw alerts is challenging due to the complexity of business logic. For example, “asset device scanning” and “port scanning attacks” may trigger identical alert types, making it difficult to distinguish legitimate activities from actual threats. Without advanced methods to analyze

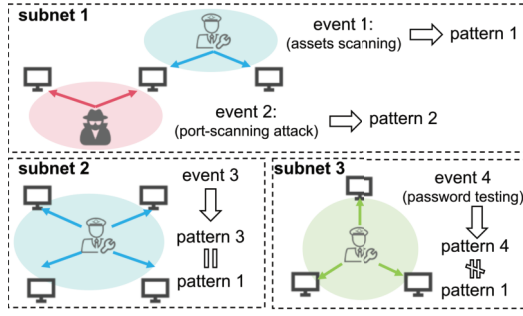


Fig. 1. Example of correlated alerts in a real enterprise intranet. Arrows in different colors represent different types of alerts. The example covers three subnets and four security events, where events 1 and 3 share the same pattern and belong to the same category of security events.

the context and relationships between alerts, SOC operators must manually investigate each alert, leading to inefficiency and potential oversight of critical issues.

To address these challenges, this paper introduces a pattern recommendation approach that accurately reconstructs security events from raw alerts and recommends corresponding patterns. For example, in Fig. 1, related alerts are grouped into four security events by correlation features, including spatial correlation (e.g., relationships between IP addresses and ports), temporal correlation (e.g., alerts occurring within the same time window), and alert categories (e.g., identifying whether alerts correspond to scanning, brute-force attacks, or anomalous traffic). Then, event 1 and event 3 are classified in the same pattern, as their features are highly similar correlation features. By clustering similar security events into the same pattern, SOC operators can focus on analyzing a small subset of representative security events for each pattern, significantly reducing the workload.

III. METHODOLOGY

We propose a method based on event reconstruction and pattern recommendation, DEEPDRAC, which helps operators quickly dispose of alerts. The framework of DEEPDRAC is shown in Fig. 2, which consists of three components: the Event Graph Builder for reconstructing security events (Section III-B), the Pattern Generator for generating patterns (Section III-C), and the SemiAutomatic Decision for alert disposal by recommending patterns (Section III-D). Notably, the Pattern Generator uses the GNN to extract the semantic details of security events. We employ a self-supervised learning approach to alleviate the dependency on labeled samples during GNN training (Section III-E).

A. Data Preprocessing on Alerts

As the data fields of alerts from different security devices are not unified and there are large amounts of duplicate alerts, we need to preprocess alerts before analysis.

First, we unify the data fields of all alerts referring to the alert fields of popular open-source network IDSs (e.g., Snort and Suricata). Taking the alert generated by Snort 3.1.18 as an example, it includes 44 fields in JSON format, whose main

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Descriptions
$\mathcal{G}$	The alert graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A}_N, \mathbf{A}_E, \mathbf{A}_G, T)$ is a directed graph constructed from alerts generated within a specified time window T .
$\mathcal{V}$	The set of nodes in graph $\mathcal{G}$.
v_i	A node $v_i \in \mathcal{V}$.
$\mathcal{E}$	The set of edges in graph $\mathcal{G}$.
e_{ij}	An edge $e_{ij} \in \mathcal{E}$ between node v_i and node v_j .
$\mathbf{A}_N$	The feature matrix of the nodes in graph $\mathcal{G}$.
$\mathbf{A}_E$	The feature matrix of the edges in graph $\mathcal{G}$.
$\mathcal{G}_s$	The security event, defined as a specific type of alert cluster composed of correlated alerts triggered by the same attack event or business operation. It is formally represented as a subgraph obtained through partitioning of the alert graph $\mathcal{G}$.
$\mathbf{A}_G$	The feature vector of the security event $\mathcal{G}_s$ at the graph level.
$\mathcal{G}_{embed}$	The embedding generated by the GNN for the security event $\mathcal{G}_s$.
$\mathcal{G}_{pattern}$	The security event pattern, defined as a semantically meaningful vector representation that captures the critical information of a security event. Security events sharing the same pattern typically originate from the same type of triggering behavior—for example, both “10 port scans” and “100 port scans” correspond to a common pattern of port scanning.
$\mathcal{P}_c$	The pattern cluster, defined as a collection of security events that share the same pattern, typically indicating the same triggering causes and requiring the same disposal strategies.

data fields are shown in Table II. The semantic information contained in these data fields can be classified into three types: 1) Information about the network activity, including the timestamp, the source IP address (src_addr), the destination IP address (dst_addr), and the protocol (proto); 2) Information about the activity’s traffic, including the packet length (pkt_len), the packet number (pkt_num), the TCP sequence number (tcp_seq), and the TCP window size (tcp_win); and 3) Anomaly information of the alert generated by the IDS, including the matched rule ID (rule), the anomaly category (category), the anomaly message (message), etc. In this work, we select a subset of the data fields (i.e., the highlighted fields in Table II), which is important for alert analysis and available in most security devices.

Second, for alerts that have timestamps within the same time window T , if the values of the fields (src_addr, dst_addr, dst_port, proto, rule, category and priority) are identical, they are premerged to reduce duplicate alerts. The number of merged alerts is denoted as warn_num, and the alert fields used for constructing the alert graph are given by (timestamp, src_addr, dst_addr, dst_port, proto, rule, category, priority, and warn_num). Here, the timestamp refers to the start time of the time window T .

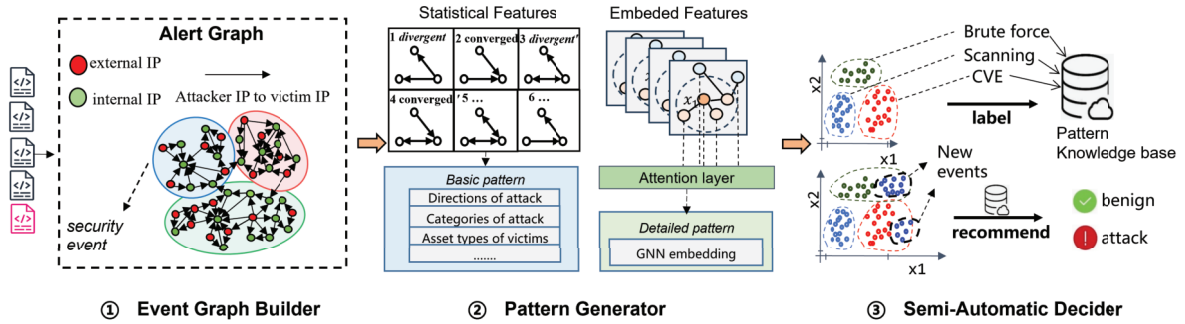


Fig. 2. Framework of DEEPDRAC that shows the three main components. ① Event graph builder: An alert graph is built based on the alerts where the edge denotes alerts from one host (i.e., the node in the alert graph) to another host. Then, a community detection algorithm is applied to the alert graph, generating security events that contain contextual information. ② Pattern Generator: Patterns of security events are generated based on statistical features and GNN embeddings. On the one hand, statistical features are extracted and discretized to form the basic pattern, which intuitively describes the security event. On the other hand, GNN embeddings capture detailed features as supplements, together forming a comprehensive pattern. This pattern encapsulates the critical information of the security event. ③ Semiautomatic decoder: First, security events with the same patterns are grouped. Second, a small number of labeled security events are introduced to generate a pattern knowledge base, which is used to identify patterns for subsequent alerts. With this decoder, frequently occurring false alerts can be automatically filtered while maintaining good interpretability.

TABLE II

EXAMPLE OF THE MAIN FIELDS FROM THE ALERT OF SNORT 3.1.18. THE BOLDED ALERT FIELDS ARE USED BY DEEPDRAC

Field Name	Field Content
timestamp	04/02-21:00:13.929875
proto	UDP
dir	S2C
src_addr	172.16.112.20
src_port	53
dst_addr	192.168.1.10
dst_port	53
pkt_len	128
pkt_num	16
rule	1:15934:12
category	Potential Corporate Privacy Violation
priority	1
message	PROTOCOL-DNS dns response for rfc1918 172.16/12 address detected

B. Event Graph Builder

After alerts are unified and premerged, they are input into the event graph builder to mine the spatial correlation relationships between alerts.

Definition 1 (Alert Graph $\mathcal{G}$) The alert graph is a directed attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A}_N, \mathbf{A}_E, \mathbf{A}_G, T)$ formed by alerts within a time window T . Node $v \in \mathcal{V}$ represents a host with a unique IP address, indicating either a suspicious attacker or a potential victim in the network. The edge $e_{ij} \in \mathcal{E}$ represents the merged alerts from the attacking IP address i (src_addr) to the victim IP address j (dst_addr). Since the dst_port, category, and other features may differ, there can be multiple edges between two nodes, with each edge representing a specific type of network activity. $\mathbf{A}_N$ and $\mathbf{A}_E$ represent the node and edge feature matrix, respectively, whereas $\mathbf{A}_G$ denotes the graph-level features.

In one time window, alerts may contain multiple unrelated security events. The connections between alerts belonging to the same security event are close-knit, whereas the connections with alerts from other events are loose, this results in a clear community structure on the alert graph. Communities are subgraphs composed of a group of nodes and edges, which may share common features or be closely related in the graph [24]. The nodes and edges in the same community are more likely to connect. DEEPDRAC employs an enhanced Louvain algorithm based on alert categories and attempts to group alerts belonging to the same security event into one community. This community is referred to as the security event and its detailed definition is as follows.

Definition 2 (Security Event $\mathcal{G}_s$) Security event $\mathcal{G}_s$ is a specific type of alert cluster that consists of alerts triggered by the same root cause, such as a single attack event or a business operation. It is represented as a subgraph obtained through community detection on the alert graph $\mathcal{G}$ and contains contextual information of the corresponding event. Formally, $\mathcal{G}_s = \{\text{Alert}_1, \text{Alert}_2, \dots, \text{Alert}_n\}$, where each Alert_i represents an individual alert associated with the same attack event or business operation.

There are several issues when traditional community detection algorithms are applied to alert graphs. First, current community detection algorithms [25], [26], [27], [28] split the graph into multiple communities (i.e., node sets) by cutting some edges, which may split the attacker and the victim of some alerts into two communities. Second, few community detection algorithms consider node or edge features, resulting in imprecise alert communities. To solve these issues, we introduce alert categories into community detection algorithms, achieve edge-based alert graph clustering and propose the enhanced Louvain algorithm, as shown in Fig. 3.

The details of the enhanced Louvain algorithm are illustrated in Algorithm 1, which includes two steps: initial partitioning based on connectivity and correction based on alert categories. We first apply the Louvain algorithm to obtain the initial set of communities $\mathcal{C}$ of the alert graph (Line 1). We mine the frequent combinations of alert categories in $\mathcal{C}$.

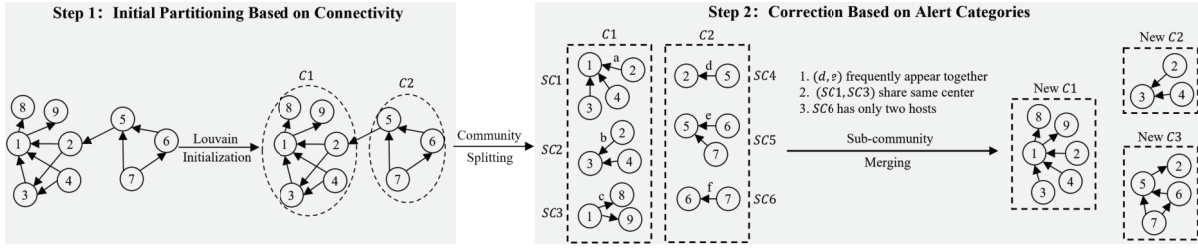


Fig. 3. Enhanced Louvain algorithm based on alert categories. Implementation consists of two steps. 1) In Step 1, the Louvain algorithm is employed to divide the alert graph, generating two communities $C1$ and $C2$ in the example. 2) In Step 2, these communities are further split based on alert categories. First, $C1$ and $C2$ are decomposed into six subcommunities $SC1 - SC6$, where each subcommunity has only one central node and the same *category* (i.e., a, b, c, d, e, f). Then, subcommunities within the same community that meet the merging rules are combined to form the final security event graph.

Algorithm 1 Enhanced Louvain Algorithm Based on Alert Categories

Input: Alert graph $\mathcal{G}$

Output: Security events $\mathcal{G}_s$

```

1  $\mathcal{C} \leftarrow$  The set of initial communities from the Louvain
  algorithm on  $\mathcal{G}$ 
2  $FP \leftarrow$  Mine frequent combinations of alert categories in
   $\mathcal{C}$ 
3 for community  $C \in \mathcal{C}$  do
4   if  $C$  is the single-center topology of the same alert
     category then
5     Leave  $C$  unchanged
6   else
7     Select node  $v$  with the highest degrees
8     Collect alert categories, src_addr, and dst_addr
       related to  $v$ 
9     while highest degree of remaining nodes  $> 2$  do
10      Identify alerts with the same attack direction and
        the highest number of categories.
11      Split them into subcommunities
12    end while
13    for frequent combination  $p \in FP$  with high confi-
       dence do
14      Merge subcommunities containing frequent com-
        bination  $p$ 
15    end for
16    for subcommunity  $SC$  with the same central node
       and a category do
17      Merge  $SC$  into one community
18    end for
19    for subcommunity  $SC$  with only two hosts do
20      Merge  $SC$  with an adjacent subcommunity
21    end for
22  end if
23 end for
24 return:  $\mathcal{G}_s$ 

```

which is called the **central node**. Finally, we merge some of the subcommunities according to the preset rules to obtain the community result: security events $\mathcal{G}_s$ (Lines 13–23). The specific principle of the algorithm is as follows.

1) *Community Initialization Based on Connectivity*: To divide the original alert graph into different communities, we initially employ the Louvain algorithm, which is one of the most widely used community detection algorithms and is based on modularity maximization [27]. The modularity measures the density of edges inside communities compared with edges between communities, which can be defined as:

$$Q = \frac{1}{2m} \sum_{i,j} \left[W_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (1)$$

where m represents the sum of all of the edge weights in the graph, W_{ij} denotes the weight between node i and node j , k_i and k_j represent the sum of the weights of the edges attached to v_i and v_j , respectively. c_i and c_j are the communities to which v_i and v_j belong, respectively. δ denotes the Kronecker delta function, where $\delta(c_i, c_j)$ is 1 if c_i and c_j are the same community, and 0 otherwise. A larger Q indicates a more compact community structure after aggregation. The Louvain algorithm seeks to divide the graph into communities by maximizing the modularity Q on the basis on a greedy strategy.

2) *Community Correction Based on Alert Categories*: Some communities generated by the Louvain algorithm exhibit complex topologies, with numerous alerts intertwined, making it difficult to accurately reflect the information of a single security event. Therefore, it is necessary to further correct the clustering results.

We find that 82.71% of security events consist of alerts of the same category and exhibit a single-center topological type when analyzing real-world datasets. Based on the above analysis, for single-center communities of the same alert category identified by the Louvain algorithm, we consider the clustering results to be satisfactory and leave them unchanged. For other communities with more complex topologies, several scenarios may guide us in optimizing the complex topological communities:

- Some sporadic alerts link multiple originally unrelated alert communities together.
- The attacker triggered multiple categories of alerts via various attack methods.

The results are used in the subsequent algorithm (Line 2). After that, we iterate through each community in the set $\mathcal{C}$ and split the community into subcommunities with the single-center topology of the same alert category (Lines 3–12). In this context, a single-center topology denotes a community with exactly one node directly connected to all other nodes,

- Complex normal network activities trigger various categories of false positive alerts.

To improve the precision of community detection, we first apply the FP-tree algorithm [29] to discover frequent combinations of alert categories in the Louvain algorithm's clustering results. For example, alerts indicating exploitation behaviors of different common exposures and weaknesses (CVEs) may appear simultaneously, which are attack attempts before a successful invasion. The temporal correlation between these behaviors indicates a high-security threat. Hence, these alerts should be clustered into the same community. Based on the results of frequent combination mining, we refine the multicenter topologies in the Louvain algorithm's clustering results as follows:

- 1) We first divide each community obtained from the Louvain algorithm into subcommunities with single-center topology and the same alert category.
- 2) Based on the frequent combinations of alert categories, we merge subcommunities whose categories often appear together. These highly-confident combinations may indicate a highly threatening network attack or a specific normal network activity.
- 3) We merge single-center topology subcommunities of the same alert category with the same central node. Single-center topology communities of the same alert category with the same central node might represent multiple attack attempts by the same attacker, and the resulting topology after merging is simpler and easier to analyze.
- 4) We merge subcommunities containing only two hosts with their adjacent subcommunities. As subcommunities with only two hosts lack contextual information for alerts and are difficult to analyze effectively, we merge them with adjacent subcommunities.

By incorporating domain-specific knowledge alongside the topological structure of the alert graph, the enhanced Louvain algorithm improves the traditional Louvain algorithm in two main aspects. First, a decoupling mechanism is introduced based on alert categories to avoid clustering semantically unrelated alerts. Second, a merging mechanism informed by frequent co-occurrence patterns captures behaviorally correlated alerts that tend to occur together in practice. With these improvements, we can produce community partitions that better align with real-world security event boundaries, thereby addressing the tendency of the traditional Louvain algorithm to overfit purely structural connections.

C. Pattern Generator

Definition 3 (Security Event Pattern $\mathcal{G}_{pattern}$) The security event pattern (abbreviated as pattern) is a vector with semantic information, representing the critical information of the security event, which is defined as $\mathcal{G}_{pattern} = (center_num, central_ip_label, nonCentral_ip_label, topo_type, category, \mathcal{G}_{embed})$.

The security event pattern consists of two parts: the basic pattern derived from statistical features and the detailed pattern represented by GNN embeddings $\mathcal{G}_{embed}$. The basic pattern provides a rough description of critical information about the

TABLE III
FEATURES OF SECURITY EVENT $\mathcal{G}_s$

Module	Feature	Description
Node v_i	<i>high_degree_label</i>	Nodes with the highest degrees in the graph, selected such that the sum of their degrees is at least 50% of the total degree sum.
	<i>ip_label</i>	Asset types of v_i , such as DNS server, file server, etc.
Edge e_{ij}	<i>dst_port</i>	Destination port.
	<i>proto</i>	Alert protocol type.
	<i>category</i>	Alert category.
	<i>priority</i>	Priority of the alert category.
Graph $\mathcal{G}_s$	<i>warn_num</i>	The number of merged alerts.
	<i>center_num</i>	The number of nodes with <i>high_degree_label</i> = 1.
	<i>motifs</i>	Distribution of 13 three-node motifs.

security event, such as the number of primary targets involved in the attack. On the other hand, $\mathcal{G}_{embed}$ captures contextual semantic and structural information on $\mathcal{G}_s$, enhancing the expressiveness and distinguishability of $\mathcal{G}_{pattern}$.

1) *Basic Pattern*: The security event $\mathcal{G}_s$ is a subgraph composed of alerts, with its main features coming directly from alert attributes or their statistical summary, as shown in Table III. The basic pattern is derived from the transformation of these features and provides fundamental information about $\mathcal{G}_s$. The edge features are directly from the alerts, and here we discuss only the node-level and graph-level features. The features of node $v_i \in \mathcal{V}$ include *ip_label_i* and *high_degree_label_i*, where *ip_label_i* indicates the asset type associated with v_i , and *high_degree_label_i* indicates whether v_i holds a significant degree in $\mathcal{G}_s$.

A high-degree node is defined as a node whose degree is relatively large compared with other nodes in the graph. Specifically, nodes are labeled “high-degree” if their degrees collectively account for at least 50% of the total degree sum of $\mathcal{G}_s$. Nodes with *high_degree_label_i* = 1 are referred to as central nodes, which play a crucial role in network connectivity and information propagation.

At the graph level, the features include *center_num*, which represents the number of central nodes, and the distributions of three-node *motifs* within the graph. To analyze the topological characteristics of $\mathcal{G}_s$, we examine the distribution of these motifs. As defined in [30], motifs are small recurring subgraphs that occur significantly more often in $\mathcal{G}_s$ than in random graphs. These motifs provide insights into the structural properties of $\mathcal{G}_s$, such as attack directions or clustering patterns. We use the FANMOD algorithm [31] to mine 13 three-node motifs (as shown in Fig. 4) and calculate their percentage in $\mathcal{G}_s$.

The basic pattern is derived from the discretization and classification of features and is specifically defined as follows: (*center_num*, *central_ip_label*, *nonCentral_ip_label*, *topo_type*, *category*). *center_num* represents the number of central nodes in $\mathcal{G}_s$, and *central_ip_label* is the set of *ip_label* corresponding to these central nodes. *nonCentral_ip_label* is the set

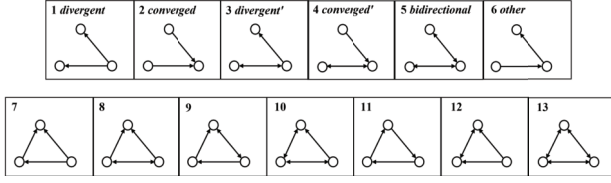


Fig. 4. 13 types of three-node motifs. If the proportion of motifs of types 1–5 is greater than τ_{motif} , the security event is defined as the corresponding topology type; otherwise, it is the other type. *divergent'* and *converged'* are weakened (or generalized) forms of *divergent* and *converged*, respectively.

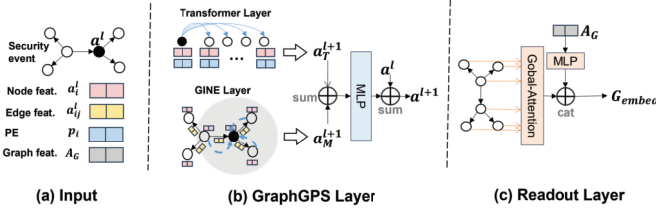


Fig. 5. Workflow for generating detailed patterns via the GNN. (a) Inputting the security event and generating position encodings (PEs) via random walks; (b) Combining the Transformer layer and GINE layer for message propagation to update node states, with this module stacked for L layers; (c) Aggregating node states based on a global attention mechanism and concatenating the projected graph features to generate $\mathcal{G}_{embed}$.

of *ip_label* corresponding to noncentral nodes. *topo_type* describes the topology type of the security event $\mathcal{G}_s$. If $\mathcal{G}_s$ has only two nodes, i.e., $|\mathcal{V}| = 2$, then the topology type is defined as simple. If $|\mathcal{V}| \geq 3$, it is determined based on the proportion of three-node motifs. As shown in Fig. 4, if the proportions of motifs 1–5 are greater than the threshold τ_{motif} , *topo_type* correspond to “diverging type, converging type, weak diverging type, weak converging type, and bidirectional type”, respectively. Otherwise, *topo_type* is defined as other types. *category* is the set of alert categories for all alerts in $\mathcal{G}_s$. If the cardinality of this set exceeds three, it is defined as multicategory. The basic pattern roughly describes the topological structure (e.g., number of central nodes, attack direction) and the attributes (e.g., types of assets associated with the main IPs, attack types) of the security event $\mathcal{G}_s$, providing fundamental information about $\mathcal{G}_s$.

2) *Detailed Pattern Based on GNN Embedding*: Motivated by the ability of machine learning technologies to identify complex and subtle pattern information [23], we employ the GNN model [32] to embed the security event $\mathcal{G}_s$, aiming to generate a detailed pattern for describing $\mathcal{G}_s$. As shown in Fig. 5, the process of embedding $\mathcal{G}_s$ can be roughly divided into three steps as follows.

a) *Input*: The security event $\mathcal{G}_s$ contains node features $\mathbf{a}_i \in \mathbf{A}_N$, edge features $\mathbf{a}_{ij} \in \mathbf{A}_E$, and graph features $\mathbf{A}_G$, which serve as the inputs of the GNN model. Additionally, because our GNN module includes a Transformer component, to enable it to sense the positional relationships between nodes, we generate node Position Encodings (PE) $\mathbf{p}_i \in \mathbf{P}$ based on random walks [33]. Then, we concatenate the node features with the projected PE, incorporating the positional information into the node features as:

$$\mathbf{a}_i^l = \mathbf{a}_i^l \oplus \Theta_0 \mathbf{p}_i \quad (2)$$

where Θ_0 is a learnable parameter implemented through a linear layer.

b) *Update Node States Based on GraphGPS*: We design a GNN convolutional layer based on the GraphGPS framework [34] to update node states, which includes a graph Transformer layer and a GINE layer [35]. Most traditional GNNs (such as GCN [36] and GAT [37]) are designed for node-centric tasks, and edge features are usually ignored in message propagation and aggregation [38]. However, in the security event $\mathcal{G}_s$, the edges contain important alert attribute information (such as destination port, anomaly category, etc.), which plays an irreplaceable role. In addition, traditional GNNs suffer from the problem of oversmoothing, which limits the model capacity. Therefore, we design a convolutional layer based on the GraphGPS framework, which combines a GINE layer sensitive to edge features and a graph Transformer layer that alleviates the oversmoothing problem. The convolution is calculated as follows:

$$\mathbf{a}_{iT}^l = \text{Transformer}(\mathbf{a}_i^l) \quad (3)$$

$$\mathbf{a}_{iM}^l = \Theta_1 \left(\mathbf{a}_i^l + \sum_{j \in \mathcal{N}(i)} \text{ReLU}(\mathbf{a}_j^l + \mathbf{a}_{ji}) \right) \quad (4)$$

$$\mathbf{a}_i^{l+1} = \mathbf{a}_i^l + \Theta_2(\mathbf{a}_{iT}^l + \mathbf{a}_{iM}^l) \quad (5)$$

where $\mathbf{a}_i^l$ is the state of node $v_i \in \mathcal{V}$ after the l -th layer of the GraphGPS convolution operation, $L \geq l \geq 0$. $\mathcal{N}(i)$ is the set of one-hop neighbors of v_i . Θ_1, Θ_2 are both linear layers.

c) *Readout the Security Event Embedding*: After L layers of node updates via GraphGPS convolution, we aggregate the information from all nodes in the graph via a global attention mechanism and combine it with the graph features to generate the security event embedding $\mathcal{G}_{embed}$. Because not all security events are perfect, some might include irrelevant nodes, and the influence of nodes in a security event is not uniform. Therefore, we employ the attention mechanism to assign weights to different nodes to measure their influence and filter out irrelevant nodes during the readout stage [39].

$$\mathcal{G}_{readout} = \sum_{v_i \in \mathcal{V}} \text{softmax}(\Theta_q(\mathbf{a}_i^L)^\top) \odot \Theta_v \mathbf{a}_i^L \quad (6)$$

where $\Theta_q, \mathbf{a}_i^L, \Theta_v \mathbf{a}_i^L$ correspond to the query, key, and value vectors in the attention mechanism, respectively. Specifically, Θ_q and Θ_v are learnable parameters. Θ_q is used to compute the attention coefficients, learning which nodes are relevant to the current task and filtering out irrelevant information. Θ_v projects the final layer node states $\mathbf{a}_i^L$ into a suitable space to better generate the graph embedding. $\odot$ denotes the Hadamard product, and the softmax function normalizes the attention coefficients. Through node updates and global aggregation, $\mathcal{G}_{readout}$ contains the node features, edge features, and graph structure information of the security event, but it still lacks the graph feature $\mathbf{A}_G$. Therefore, we project $\mathbf{A}_G$ into the feature space of $\mathcal{G}_{readout}$ and then concatenate to generate the final security event embedding.

$$\mathcal{G}_{embed} = \mathcal{G}_{readout} \oplus \Theta_{out} \mathbf{A}_G \quad (7)$$

where Θ_{out} is a learnable parameter, implemented through a linear layer.

By aggregating information from neighboring nodes and edges, the detailed patterns generated via GNN embeddings effectively capture global correlations and latent semantics among alerts. This structure-aware capability allows security events with different patterns to form clearer boundaries in the embedding space, thereby improving the clustering model's ability to differentiate them. To leverage both semantic breadth and resolution, we represent each security event via both basic patterns and GNN-based detailed patterns: the basic patterns define a coarse-grained semantic space, whereas the detailed patterns refine distinctions among similar cases. The integration of both yields more accurate and semantically coherent clustering of security events.

D. Semiautomatic Decider

Definition 4 (Pattern Cluster $\mathcal{P}_c$) Pattern cluster $\mathcal{P}_c$ is a collection of security events that share the same pattern. Each security event within the cluster exhibits similar semantic characteristics and originates from a common root cause, such as repeated instances of the attack category. Formally, a pattern cluster is defined as $\mathcal{P}_c = \{\mathcal{G}_{s1}, \mathcal{G}_{s2}, \dots, \mathcal{G}_{sn}\}$, where each $\mathcal{G}_{si}$ denotes a security event that shares the same pattern.

In the previous discussion, we obtained the pattern $\mathcal{G}_{pattern}$ for each security event $\mathcal{G}_s$. In fact, many security events triggered by the same cause do not occur just once but often appear repeatedly in a fixed pattern. We expect to aggregate security events with a consistent pattern together for batch processing. We relax the definition of security events with the same pattern to “same basic pattern, similar detailed pattern” because even security events triggered by the same cause can exhibit subtle differences in their patterns, such as 10 port scans versus 11 port scans. Nevertheless, security events with the same pattern share at least the same disposition method and risk level. In the implementation, we first use the basic pattern to divide the security event space and filter out subspaces containing fewer than N_{basic} security events to ensure consistency and frequency in the basic pattern. Second, within each subspace defined by the basic pattern, we apply DBSCAN to cluster the GNN embeddings of security events, generating pattern clusters $\mathcal{P}_c$. $\mathcal{P}_c$ is a set composed of security events, all of which share the same pattern. After that, we label pattern clusters to create a pattern knowledge base (manual analysis). Finally, new incoming alerts are organized into security events and matched with the corresponding recommended disposition methods based on the identified patterns (automated analysis).

1) *Stage 1—Manual Analysis:* In this stage, to build the initial pattern knowledge base, we incorporate a small amount of expert knowledge to label the risk levels and disposal information of pattern clusters. To ensure the accuracy of pattern knowledge, we sample multiple security events from each $\mathcal{P}_c$ for labeling.

In the ideal case, all security events in a pattern cluster $\mathcal{P}_c$ represent the same pattern, and analyzing a single security event from $\mathcal{P}_c$ is sufficient to label the entire cluster. However, there is always the possibility of errors in all the models. DEEPDRAC employs a sampling approach to reduce the possibility of model errors. DEEPDRAC expects operators to analyze N_s security events, which are sampled

without replacement of each $\mathcal{P}_c$. If their patterns are the same, the entire cluster is labeled at once, with the labeling information including the pattern description and the corresponding disposal method. If different risk levels of security events are discovered during the sampling process, all security events in the cluster will be handed over to operators for processing. In practice, the risk level of a pattern cluster is assessed according to the nature of its constituent security events: pattern clusters triggered by benign configurations or routine operations (e.g., internal scanning, whitelisted IPs) are labeled false-positives; clusters caused by nonmalicious external behaviors such as low-frequency scans are labeled low risk; those involving clearly malicious but early-stage attacks such as Nmap scans and SQL injection attempts are marked medium risk; and clusters where the attacker has gained internal access and conducted harmful actions (e.g., such as executing unauthorized code) are labeled high risk. The more samples we select, the more accurate the labeling results will be. A detailed discussion on the number of samples is presented in Section IV-B1b. Based on the capabilities of our approach, almost all security events can be handled through a small number of sampling analyses, significantly reducing the workload of operators. Manually labeled results are stored in the pattern knowledge base, providing the foundation for the subsequent automated analysis stage.

2) *Stage 2—Automated Analysis:* In this stage, the pattern knowledge base generated during the manual analysis stage is used to automatically recommend risk levels and disposition information for security events. Specifically, when a new security event enters the decider, it undergoes incremental clustering to match known pattern clusters. If a match is found, its risk level and disposition information are obtained, and an automatic response action is performed (such as filtering false positives, reporting high-risk hosts, etc.). If no matched pattern clusters are found, the security event is temporarily stored, awaiting the arrival of other security events to potentially form a new pattern cluster. Operators can set the allowed waiting time for generating a new pattern cluster. After this time, if no match is found with an existing pattern cluster and no new pattern cluster is formed, the security event will be considered an emergency and prioritized for pushing to the operators.

E. Training of the GNN

To alleviate the dependency on many labeled samples, we adopt a pretraining and fine-tuning (P&F) strategy for training the GNN [40]. In the pretraining stage, we employ a contrastive learning paradigm for self-supervised training of the GNN model. The primary goal of contrastive learning is to maximize the agreement of two jointly sampled positive pairs $(\mathcal{G}_{s_i}, \mathcal{G}_{s_j})$ and minimize the agreement of two independently sampled negative pairs $(\mathcal{G}_{s_i}, \mathcal{G}_{s_k})$ [41]. In the fine-tuning stage, the pretrained parameters are used as the initial parameters of the GNN, which is then fine-tuned with a supervised task.

1) *Pretraining of the GNN:* We generate two types of positive and negative sample pairs, which are based on the basic pattern and the graph kernel method, respectively. We generate many security events via historical data and divide them into two sets: DB_{base} and DB_{search} . For each sample

$\mathcal{G}_{s_i} \in DB_{base}$, we generate two types of positive and negative sample pairs. The first type is based on whether the samples $\mathcal{G}_{s_i}$ share the same basic pattern. A positive sample $\mathcal{G}_{s_{j1}}$ with the same basic pattern and a negative sample $\mathcal{G}_{s_{k1}}$ with a different basic pattern are sampled from DB_{search} . For the second type, we employ a graph kernel method to generate positive and negative samples inside a single basic pattern space, enhancing the GNN model's capacity to capture subtle differences. Specifically, we use the Neighborhood Subgraph Pairwise Distance Kernel (NSPDK) method [42] to compute the graph kernel distance for the sample pair $(\mathcal{G}_{s_i}, \mathcal{G}'_{s_i})$, where $\mathcal{G}'_{s_i} \in DB_{base}$ and shares the same basic pattern as $\mathcal{G}_{s_i}$. The NSPDK computes the graph kernel distance by comparing the subgraph node and edge encodings in the sample pair and is an efficient graph kernel method. We calculate the graph kernel distance for each basic pattern in DB_{base} and generate the probability density distribution. The lowest point of the probability density distribution in the middle part (0.4–0.6) implies a transition from similar to dissimilar sample pairs and is more suitable as a threshold than 0.5. Therefore, we adopt this strategy to generate the threshold ρ for each basic pattern. Then, we randomly select samples from DB_{search} with the same basic pattern as $\mathcal{G}_{s_i}$ and compute the graph kernel distance. Pairs with a graph kernel distance below ρ are considered positive sample pairs, whereas those above ρ are considered negative sample pairs.

After positive and negative sample pairs are obtained through the two aforementioned methods, we train the GNN via triplet loss [43]:

$$\mathcal{L}_{\text{triplet}} = \frac{1}{m} \sum \max \{0, d(\mathcal{G}_{s_i}, \mathcal{G}_{s_j}) - d(\mathcal{G}_{s_i}, \mathcal{G}_{s_k}) + \gamma\} \quad (8)$$

where $\mathcal{G}_{s_i} \in DB_{base}$ is the anchor graph, $\mathcal{G}_{s_j} \in DB_{search}$ is the positive sample, $\mathcal{G}_{s_k} \in DB_{search}$ is the negative sample, m is the number of triplets in the dataset, and $\gamma > 0$ is the margin parameter. $d(\cdot, \cdot)$ is a similarity measure that uses the Euclidean distance as follows:

$$d(\mathcal{G}_{s_i}, \mathcal{G}_{s_j}) = \|\mathcal{G}_{\text{embed_}i} - \mathcal{G}_{\text{embed_}j}\|^2 \quad (9)$$

where $\mathcal{G}_{\text{embed_}i}$ and $\mathcal{G}_{\text{embed_}j}$ are the GNN embeddings of $\mathcal{G}_{s_i}$ and $\mathcal{G}_{s_j}$, respectively.

2) *Fine-Tuning of the GNN*: Although pretraining allows learning the features and structural information of the security event $\mathcal{G}$, it may miss some information relevant to downstream tasks. Thus, we fine-tune the GNN parameters via a limited set of labeled security events by conducting a classification task, which identifies the risk levels of security events (i.e., false-positives, low-risk, medium-risk, and high-risk events). Specifically, the pretrained GNN parameters are used as the initialization parameters of the GNN and are jointly trained with the prediction head consisting of a linear layer and a softmax function. Because false-positive and low-risk events are much more common than other risk levels in security events, we use a cost-sensitive cross-entropy loss function to address the class imbalance issue.

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N_b} \sum_{i=1}^{N_c} \sum_{j \in s_i} c_i \cdot \log(\hat{y}_{ij}) \quad (10)$$

where N_b, N_c are the number of security events and number of classes in a batch, respectively. c_i is the cost coefficient of class i , which is the inverse of the proportion of class i samples in the total samples. s_i is the set of samples predicted by the model as class i , and $\hat{y}_{ij}$ is the predicted probability of sample j being in class i .

IV. EVALUATION

In this section, we first present our experimental settings in Section IV-A. We evaluate DEEPDRAC and compare it with three baseline methods by answering three research questions.

RQ1: *Can our approach effectively and interpretably reduce the workload of alert analysis?* (Section IV-B)

RQ2: *Can our approach remain effective with a few labeled samples, without relying on many labeled samples?* (Section IV-C)

RQ3: *Can our approach accurately reconstruct security events from raw alerts?* (Section IV-D)

A. Experimental Settings

1) *Study Setup*: Our experiments are conducted on a server running the Ubuntu 18.04.6 operating system and equipped with an Intel Xeon 3.00 GHz CPU, NVIDIA GeForce RTX 3090 GPU with CUDA 11.1, and 1024 GB of memory. The hyperparameter settings are determined as follows. As most security event graph diameters are less than 2, we adopt two GraphGPS layers (i.e., $L = 2$). Each GraphGPS layer takes as input the projected node features (30 dimensions), positional encodings (walk length = 2), and projected edge features (30 dimensions), and outputs updated node states (30 dimensions). After L layers of GraphGPS convolution, all node states are aggregated to obtain a 30-dimensional graph-level representation $\mathcal{G}_{\text{readout}}$, which is concatenated with a 2-dimensional projected graph feature to form the final 32-dimensional GNN embedding $\mathcal{G}_{\text{embed}}$. We trained the model for 100 epochs via the Adam optimizer, with a learning rate of 0.001 for pretraining and 0.00001 for fine-tuning.

2) *Evaluation Datasets*: We evaluate the performance of DEEPDRAC in alleviating alert fatigue via three datasets: a real-world dataset from a power grid company and two publicly available and widely used benchmark datasets (i.e., the DARPA'99 dataset [44] and CIC-IDS2017 dataset [45]).

a) *EIPGC Dataset*: This dataset consists of 182,640 alerts from the Enterprise Intranet of a Power Grid Company. There are approximately 3,000 physical hosts and 1,500 virtual hosts in the enterprise network, with an average traffic rate of 1.2 GB/s and a maximum of 1.9 GB/s. Traffic monitoring probes are deployed at more than 20 critical branch nodes, generating over 50,000 alerts per day, with an initial false-positive rate exceeding 90%. These alerts are aggregated and forwarded to the enterprise's SOC for further disposal. To ensure data quality, we manually labeled most alerts in this dataset based on their attributes, and these labels were further verified by SOC operators, resulting in a total of 170,710 (93.47%) labeled alerts. The remaining 11,930 (6.53%) alerts are unlabeled due to insufficient alert information.

The labeled data include various security events with different risk levels. Among them, 38,817 (22.74%) false-positive

alerts are triggered by normal business, such as internal vulnerability scanners and video conferences; 112,972 (65.18%) low-risk alerts are triggered by suspicious behaviors, such as port scans and DNS server version queries; 13,336 (7.30%) medium-risk alerts are triggered by behaviors, such as file detection and vulnerability exploitation, and 5,585 (3.06%) high-risk alerts like WebLogic command execution vulnerability and the exploitation of common exposures and weaknesses (CVEs).

b) DARPA'99 Dataset: This dataset is a mixture of normal and real attack traffic in the network testbed of the Air Force Research Laboratory (AFRL), covering 58 typical attack methods that can be classified into five major categories, including scanning and probing (probe), denial of service (DOS), remote to local (R2L), user to local (U2L), and data exfiltration (Data). As one of the most comprehensive attack testing datasets, it has been widely used in the research community as a benchmark dataset. In this work, we utilize the traffic data from the last two weeks of the DARPA'99 dataset and generate alerts via Snort 3.1.18. The alerts are then labeled via the ground truth provided in the dataset, resulting in a total of 585,925 fully labeled alerts. Although the DARPA'99 dataset does not provide risk levels for each attack event, for subsequent risk level experiments, we classify all alerts triggered by attacks as high-risk and those triggered by normal traffic as false-positives.

c) CIC-IDS2017 Dataset: The CIC-IDS2017 dataset, released by the Canadian Institute for Cybersecurity (CIC), was generated by simulating realistic background traffic and recent network attacks such as Brute Force FTP and DoS. It contains five days of traffic in pcap format, with ground truth labels provided based on flow 5-tuples and timestamps. We convert the 59 GB of traffic into 685,560 alerts via Snort 3.1.18. Except for Heartbleed, Snort successfully detects the remaining 13 categories of attacks. Among them, the most prevalent alert types are DoS Hulk and false positives, accounting for 50.95% and 28.98% of all alerts, respectively. For consistency, we assign risk levels to alerts following the labeling strategy used in the DARPA'99 dataset.

The three datasets are divided into two parts: a training set for training GNNs and a test set for evaluating the effectiveness of automatic alert filtering. For the EIPGC dataset, we use the first two days with a total of 86,398 alerts as the training set and the last day with 41,403 alerts as the test set. For the DARPA'99 dataset, we use a total of 165,795 alerts in the fourth week as the training set and a total of 86,007 alerts on the first two days of the fifth week as the test set. For the CIC-IDS2017 dataset, since each category of attack occurs only during a specific time period and does not reappear later, we sort all alerts chronologically and use the first 70% of each attack category as the training set and the remaining 30% as the test set.

3) Baseline Defense Methods: We use two typical alert reduction methods and the incomplete DEEPDRAC (i.e., non-GNN, Non-Pretrained, Unsupervised) as baselines.

a) Alert Throttling: In the context of alert throttling, when an event is repeatedly triggered within a specific time frame, only one consolidated alert is presented to the operator.

For example, if the *equivalent alert Y* is triggered 10 times during the throttling interval, the operator will only receive a single alert after the first occurrence. *Equivalent alert Y* refers to the alert where all key fields are the same. All subsequent instances of *Equivalent alert Y* within that period are merely appended as supplementary details to the initial alert.

b) DeepCASE: Unlike traditional methods, such as alert throttling, DeepCASE is a state-of-the-art (SOTA) method for alert reduction based on deep learning [12]. DeepCASE is a semi-supervised learning method based on alert sequences, which mainly includes four modules. First, the SEQUENCING EVENTS MODULE collects and analyzes security events to identify relevant contextual events. The CONTEXT BUILDER module uses a deep learning model with an attention mechanism to identify relevant contextual events and build an attention vector. Next, the INTERPRETER module groups the sequences into clusters based on total attention for each distinct contextual event in a sequence. Finally, DeepCASE is also divided into manual analysis and automated matching stages. The manual analysis module presents each cluster of similar event sequences to an operator who decides how it should be treated. The semiautomatic analysis module compares the attention-weighted events of a new sequence to known clusters and automatically warns the operator in case of a match.

c) Ablation Study: We conduct extensive ablation experiments to validate the effectiveness of the enhanced Louvain algorithm, GNN module, and the training strategy. During the security event reconstruction phase, we compare the enhanced Louvain algorithm with the standard Louvain algorithm in terms of reconstruction accuracy and time consumption. During GNN training, we construct two versions using only pretraining (i.e., unsupervised) or fine-tuning (i.e., non-pretrained) to validate the indispensability of both stages. Additionally, we build a version where the GNN module is completely removed to validate the importance of embedding in providing detailed patterns (i.e., non-GNN). Because the non-GNN version only contains basic patterns, the semiautomatic decider differs between the two stages. In the manual analysis stage, the basic pattern is used directly as the pattern, and the highest risk level found during the sampling process is still taken as the risk level of the pattern cluster. In the automated analysis stage, the security event is matched to the pattern cluster with the same basic pattern for processing.

B. RQ1: Can Our Approach Effectively and Interpretably Reduce the Workload of Alert Analysis?

To answer RQ1, we evaluate the workload reduction effects of our approach in both the manual analysis stage and the automated analysis stage. The metrics for evaluating reduction effectiveness in the manual analysis stage include the percentage of alerts covered by the model's capability (coverage), the workload reduction rate for the covered portion (reduction), and the overall workload reduction rate, including uncovered alerts (overall). Furthermore, we evaluate performance in the automated analysis stage in terms of accuracy, recall, precision, the F1-score, and the percentage of alerts where the model underestimates the risk level (underestimation). The

TABLE IV

WORKLOAD REDUCTION RESULTS. THE AVERAGE WORKLOAD REDUCTION OF DEEPDRAC COMPARED WITH BASELINE METHODS. OVERALL REPRESENTS THE OVERALL WORKLOAD REDUCTION RATE DURING THE MANUAL ANALYSIS STAGE. UNDERESTIMATION (UNDER.) INDICATES THE PROPORTION OF ALERTS WHOSE RISK LEVELS ARE INCORRECTLY UNDERESTIMATED DURING THE AUTOMATED FILTERING STAGE. THE FORMULAS FOR OVERALL AND UNDER. ARE DEFINED BOTTOM THIS TABLE. HIGHLIGHTED ARE THE TOP FIRST (RED), SECOND (YELLOW), AND THIRD (GREEN) RESULTS

Dataset	Model	Events ^A	Overall ^B ↑	Accuracy↑	Recall↑	Attacks ^C ↑	Precision↑	F1-Score↑	Under. ^D ↓
DARPA'99	DeepCASE (original ¹)	3366	96.50%	5.59%	95.10%	524	0.80%	1.58%	0.0390%
	DeepCASE (best ²)	2431	94.95%	97.71%	91.67%	33	2.47%	4.80%	0.0052%
	Alert throttling (15mins)	37386	77.45%	96.38%	50.00%	62	4.75%	8.68%	0.1700%
	Alert throttling (30mins)	34074	79.45%	96.38%	50.00%	62	4.75%	8.68%	0.1700%
	Alert throttling (60mins)	31484	81.01%	96.38%	50.00%	62	4.75%	8.68%	0.1700%
	Alert throttling (1day)	23386	85.89%	96.38%	50.00%	62	4.75%	8.68%	0.1700%
	Ours(non-GNN)	5518	96.61%	14.99%	56.24%	1568	2.68%	5.11%	1.7820%
	Ours(Non-Pretrained)	2375	98.35%	92.13%	97.21%	522	10.02%	18.17%	0.0349%
	Ours-Unsupervised	2790	98.18%	92.98%	95.33%	510	10.73%	19.30%	0.0411%
	Ours	1694	98.78%	97.96%	97.39%	746	38.71%	55.40%	0.0339%
EIPGC	DeepCASE (original ¹)	10475	76.30%	30.43%	32.74%	555	97.04%	46.92%	0
	DeepCASE (best ²)	10960	63.85%	80.77%	85.29%	426	98.23%	90.72%	0
	Alert throttling (15mins)	82640	4.35%	94.84%	99.69%	328	94.29%	96.87%	0.033%
	Alert throttling (30mins)	82638	4.35%	94.84%	99.69%	328	94.29%	96.87%	0.033%
	Alert throttling (60mins)	82638	4.35%	94.84%	99.69%	328	94.29%	96.87%	0.033%
	Alert throttling (1day)	82638	4.35%	94.84%	99.69%	328	94.29%	96.87%	0.033%
	Ours(non-GNN)	2988	95.90%	57.66%	58.75%	193	96.48%	71.46%	0.063%
	Ours(Non-Pretrained)	1920	95.64%	95.20%	99.06%	601	96.88%	97.64%	0.0917%
	Ours-Unsupervised	2142	95.30%	94.77%	98.78%	671	96.76%	97.40%	0.0849%
	Ours	2016	95.38%	97.04%	99.22%	745	97.54%	98.25%	0.014%
CIC-IDS2017	DeepCASE (original ¹)	60189	87.46%	56.33%	84.37%	33131	54.93%	66.54%	0.0006%
	DeepCASE (best ²)	53916	88.77%	61.94%	88.06%	42309	60.76%	71.91%	0.0004%
	Alert throttling (15mins)	59853	87.53%	90.01%	100.00%	146989	88.86%	94.10%	0
	Alert throttling (30mins)	58394	87.83%	90.01%	100.00%	146989	88.86%	94.10%	0
	Alert throttling (60mins)	57552	88.01%	90.01%	100.00%	146989	88.86%	94.10%	0
	Alert throttling (1day)	55479	88.44%	90.01%	100.00%	146989	88.86%	94.10%	0
	Ours(non-GNN)	1061	99.78%	90.18%	99.88%	141330	88.41%	93.79%	0.0899%
	Ours(Non-Pretrained)	4377	99.29%	98.07%	99.97%	139694	97.96%	98.96%	0.0307%
	Ours-Unsupervised	3700	99.43%	97.78%	99.96%	136275	97.55%	98.74%	0.0334%
	Ours	4888	99.19%	98.13%	99.99%	138566	97.98%	98.98%	0.0050%

- 1: The model directly uses the parameters from the original paper.
2: The model uses the parameters that perform best in the experiments conducted in this paper.
A: The number of security events sent to the operator.
B: The total reduction, evaluated as the sampled security events + uncovered security events compared with the total alerts.
C: The number of attack alerts recalled by the model.
D: The percentage of alerts with underestimated risk levels.

Mathematical Definitions:

$$B = 1 - \frac{\text{Sampled Events} + \text{Uncovered Events}}{\text{Total Alerts}}$$

$$D = \frac{1}{\text{Total Alerts}} \sum_{i=1}^{\text{Total Alerts}} \mathbb{I}(\text{pred}_i < \text{label}_i)$$

experimental results on the DARPA'99 and EIPGC datasets are presented in Table IV. The following subsections detail how these results are obtained and provide corresponding analyses.

1) *Manual Analysis Stage*: In this subsection, we evaluate DEEPDRAC starting without any prior knowledge of the pattern clusters. In the manual analysis stage (Section III-D), DEEPDRAC reduces the workload by allowing operators to analyze a small number of security events within pattern clusters rather than examining all alerts.

a) *Coverage*: For DEEPDRAC, the coverage is the percentage of alerts contained in the pattern cluster and the two outlier types not covered by the model: (i) Those where the basic pattern contains fewer than N_{basic} security events lack generality and are more likely to have been formed by chance. (ii) Noise points generated during the initial clustering process. Setting $N_{basic} = 5$ to identify outliers, DEEPDRAC achieves coverage rates of 89.17%, 89.37%, and 95.86% on the CIC-

IDS2017, EIPGC, and DARPA'99 datasets, respectively. A detailed evaluation of the pattern clustering quality under this setting is provided in Supplementary Material Appendix A.

b) *Number of Samples*: For a single risk level pattern cluster, operators can dispose of the entire cluster by sampling any security event. However, there are also pattern clusters containing different risk levels, which we refer to as *bad pattern clusters*. For bad pattern clusters, a single sample analysis is not sufficient to address the whole cluster, but requires further investigation by the operator. Operators need to analyze a certain number of samples to identify bad pattern clusters, ensuring the reliability of the alert disposition results. The probability of successfully identifying whether a pattern cluster is a bad pattern cluster is as follows:

$$P(\text{bad}|N_s) = 1 - \sum_{r \in P_c} \frac{\binom{|r|}{N_s}}{\binom{N_r}{N_s}} \quad (11)$$

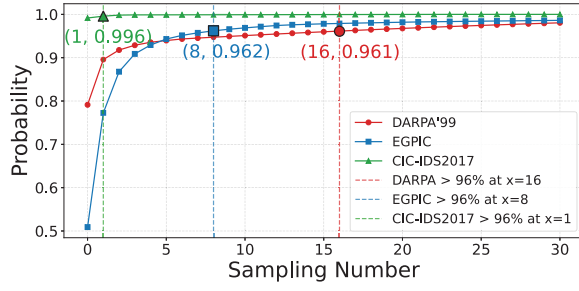


Fig. 6. Probability of identifying bad pattern clusters (pattern clusters containing security events with different risk levels), which varies with the number of samples taken by the operators for each cluster.

where N_e is the number of security events contained in a pattern cluster $\mathcal{P}_c$, $r \subset \mathcal{P}_c$ represents a set of security events within a single risk level and $|r| \geq N_s$; otherwise, $P(\text{bad}|N_s) = 1$. According to (11), the change in the probability of identifying bad pattern clusters as the number of samples increases on the EIPGC dataset is shown in Fig. 6. DEEPDRAC can successfully identify bad pattern clusters with a probability of 96.20% by sampling eight security events for each pattern cluster on the EIPGC dataset. If the number of sampling events is 18, the probability of successfully identifying bad pattern clusters reaches 98%. Additionally, in the experiments, the highest risk level is used as the risk level of bad pattern clusters, which is consistent with the way alerts are processed in reality.

c) *Workload Reduction*: We conduct a real-world simulation of DEEPDRAC. To ensure a fair comparison with other methods, we set the sampling number of all methods to the minimum value at which the probability of identifying the bad pattern cluster exceeds 96%. For the covered alerts of DEEPDRAC, the reduction is the percentage reduction in workload when alerts are disposed of based on pattern clusters:

$$\text{reduction} = 1 - \frac{N_s \times N_{\text{clusters}}}{N_{\text{cover}}} \quad (12)$$

where N_s , N_{clusters} and N_{cover} are number of security events sampled from each pattern cluster $\mathcal{P}_c$, the number of $\mathcal{P}_c$ and the number of covered alerts, respectively. DEEPDRAC achieves a reduction of 98.85%, 97.77%, and 99.75% in analysis workload on the DARPA'99, EIPGC, and CIC-IDS2017 datasets, respectively. For all alerts, the overall work reduction is measured as follows:

$$\text{Overall} = 1 - \frac{N_s \times N_{\text{clusters}} + N_{\text{uncover}}}{N_{\text{all}}} \quad (13)$$

where N_{all} is the total number of alerts, and N_{uncover} refers to the number of security events that are not covered.

As shown in Table IV, DEEPDRAC consistently achieves the highest overall reduction in alert analysis workload across all three datasets, with reductions exceeding 95%. The effectiveness of DEEPDRAC in reducing alert analysis workload primarily stems from its two-stage alert compression process: first aggregating raw alerts into security events, and then grouping these events into pattern clusters. Compared to alert throttling based on alert similarity and DeepCASE based on event sequences, the graph-based design of DEEPDRAC

captures both temporal and spatial correlations among alerts, enabling more efficient compression. This advantage results in fewer security events that require manual analysis, as evidenced by the “Events” column in Table IV. In particular, for the EIPGC dataset, which contains alerts generated in a complex network topology, DEEPDRAC demonstrates a clear advantage over DeepCASE by leveraging spatial information. This leads to a significant performance improvement, achieving a 19.08% higher overall workload reduction compared with the second-best method, DeepCASE.

2) *Automated Analysis Stage*: In the automated analysis stage (Section III-D), the newly arrived alerts are processed by DEEPDRAC to generate security events and patterns. Then, DEEPDRAC recommends disposition methods and event descriptions for the new alerts by matching with the pattern knowledge base, enabling automatic disposition. It is important to note that misclassifications in the recommendation results include both overestimations and underestimations. Although overestimating risk will reduce workload reduction, it is generally preferable to underestimating, as missing true threats can lead to more serious consequences in real-world security scenarios. Table IV lists the matching results of DEEPDRAC and the two baseline methods, from which we draw the following conclusions:

- (i) DEEPDRAC provides the most accurate recommendations for alert disposition. Across all three datasets, DEEPDRAC nearly always achieves the best performance in terms of accuracy, recall, precision, and F1 score. The only exceptions are recall on the EIPGC and CIC-IDS2017 datasets (0.47% and 0.01% below the top performer, respectively) and precision on EIPGC (0.69% lower than the top performer).
- (ii) DEEPDRAC captures more attacks within the alerts while maintaining an extremely low underestimation rate. Although DEEPDRAC does not achieve the lowest underestimation rate, it consistently maintains a low rate of no more than 0.04% across all datasets. Meanwhile, except for slightly fewer matched attack counts than Alert throttling (which has limited accuracy) on CIC-IDS2017, DEEPDRAC captures the most attacks among the three methods when matching the same number of alerts on the other two datasets. The ability to maintain an underestimation rate below 0.04% and an accuracy rate exceeding 97% while capturing more attack alerts indicates that the recommendations of DEEPDRAC are more precise and robust. In contrast, although DeepCASE attains a slightly lower underestimation rate, it captures significantly fewer attacks and exhibits much lower accuracy (80.77% on EIPGC and 61.94% on CIC-IDS2017), indicating that it tends to overestimate the severity of alerts excessively.
- (iii) The GNN module, pretraining strategy and fine-tuning strategy of DEEPDRAC are all effective and indispensable. DEEPDRAC(non-GNN) shows severe degradation, with a recommendation accuracy of only 14.99% on the DARPA'99, far below the 97.96% achieved by DEEPDRAC. Similarly, DEEPDRAC(non-pretrained)

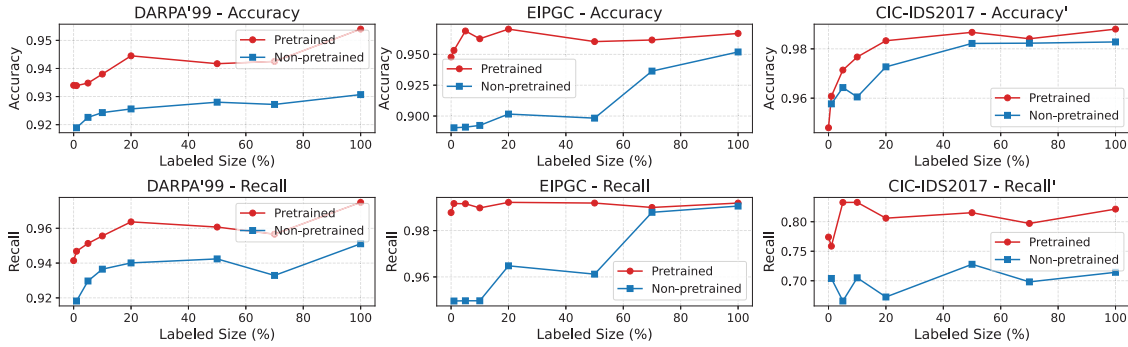


Fig. 7. Performance of DEEPDRAC (both pretrained and non-pretrained versions) when fine-tuned with labeled samples of different sizes on the DARPA'99 and EIPGC datasets. 1) DEEPDRAC has the ability for unsupervised learning. When the train data size is 0, the pretrained model achieves over 90% accuracy and recall on both datasets. 2) DEEPDRAC can learn from a few labeled samples. When the train data size is 1%, 5% and 10%, the pretrained model shows improvements in both accuracy and recall (in contrast, the non-pretrained model does not show such improvements on the EIPGC and CIC-IDS2017 datasets). 3) DEEPDRAC can quickly approach the model's performance limit. When the training data size reaches 20%, the pretrained model reaches its performance limit on the EIPGC dataset, and on the other two datasets, both its accuracy and recall are within 3% of the best results.

and DEEPDRAC-unsupervised (non-finetuned) also fail to reach the performance of DEEPDRAC.

Answer to RQ1: DEEPDRAC can effectively reduce the workload of alert analysis and significantly outperforms the two baseline defense methods. In addition to recommending disposal methods, DEEPDRAC also provides descriptions of security events, maintaining interpretability. The practical implications of DEEPDRAC for SOC operators are illustrated in Appendix B of the Supplementary material.

C. RQ2: Can Our Approach Remain Effective With a Few Labeled Samples Without Relying on a Large Number of Labeled Samples?

To answer RQ2, we evaluate the effectiveness of DEEPDRAC using different sizes of labeled samples, with a particular focus on its performance when only a few labeled samples are available. It should be noted that due to the attack alerts in CIC-IDS2017 being concentrated in several DoS security events, and both pretrained and non-pretrained models being able to effectively identify these events, the performance difference between the two models at the alert level is minimal. Therefore, we further evaluate them at the event level for the CIC-IDS2017 dataset, while retaining alert level evaluation for the other two datasets.

As shown in Fig. 7, we fine-tune DEEPDRAC using labeled samples ranging from 0% to 100% and evaluate the performance of the pretrained model and the non-pretrained model in the automated analysis stage. First, the experimental results show that DEEPDRAC has the ability for unsupervised learning. When the training data size is 0, meaning that no labeled samples are involved in the training, the pretrained model achieves accuracy and recall rates above 90% on both datasets and even approaching the performance of the non-pretrained model with 100% train data size. Since the non-pretrained model starts with randomly initialized parameters and lacks the ability to perform meaningful matching at 0%, we begin its evaluation from the 1% setting. Second, the experimental results show that DEEPDRAC can learn from a few labeled samples, whereas the non-pretrained model struggles to do so. When the training data size is less than or equal to 10%, the

non-pretrained model fails to learn meaningful information on the two more complex datasets, showing improvement only on the relatively simpler DARPA'99 dataset. In contrast, the pretrained model can effectively leverage a small number of labeled data, achieving significantly improved accuracy and recall across all three datasets with only 5% or 10% of the labeled data. Third, the experimental results show that DEEPDRAC can quickly reach the model's performance limit. When the train data size is 20%, the pretrained model reaches its performance limit on the EIPGC dataset, and on the other two datasets, both its accuracy and recall are within 2% and 3% of the best results, respectively.

Answer to RQ2: Benefiting from the pretraining strategy, DEEPDRAC remains effective with a few labeled samples and can quickly reach its performance limit as the number of samples increases without relying on a large amount of labeled data.

D. RQ3: Can Our Approach Accurately Reconstruct Security Events From Raw Alerts?

To answer RQ3, we evaluate the quality of the security events generated based on the partitioning of the alert graph. We conduct comparisons across three datasets via several community detection algorithms, including Girvan-Newman (GN), Label Propagation Algorithm (LPA), Louvain, and Leiden. Since the enhanced Louvain algorithm applies a secondary refinement over the Louvain results, we additionally include a nested application of the Louvain algorithm (denoted as D-Louvain) as a comparative baseline.

A necessary condition for alerts contained in a security event being part of the same security event is that these alerts share the same risk level. The distribution of risk levels for the security event in three datasets is shown in Table V. Across all three datasets, DEEPDRAC achieves high-quality reconstruction of security events from raw alerts: over 98% of the constructed events consist of alerts sharing the same risk level, and these events collectively contain more than 95% of all alerts. When we relax the constraint to allow grouping alerts with adjacent risk levels (e.g., low and medium, or medium and high), this percentage increases to over 96%.

TABLE V
QUALITY EVALUATION OF SECURITY EVENT RECONSTRUCTION FROM
RAW ALERTS

Dataset	Model	Events	Risk Events (S)	Risk Alerts (S)	Risk Alerts (A)
DARPA'99	GN	4474	98.10%	95.27%	95.27%
	LPA	4726	98.22%	95.54%	95.54%
	Leiden	4454	98.09%	95.32%	95.32%
	Louvain	4425	98.12%	95.51%	95.51%
	D-Louvain	5239	98.61%	96.41%	96.41%
	Ours	7813	98.85%	97.49%	97.49%
EIPGC	GN	10999	97.15%	81.36%	93.91%
	LPA	11963	97.58%	78.98%	95.20%
	Leiden	10908	97.18%	80.12%	94.52%
	Louvain	10896	97.16%	77.99%	94.52%
	D-Louvain	12579	97.98%	82.87%	95.91%
	Ours	13710	98.84%	95.25%	98.30%
CIC-IDS2017	GN	2028	98.22%	49.72%	49.72%
	LPA	1559	97.75%	55.04%	55.04%
	Leiden	2008	98.21%	49.65%	49.65%
	Louvain	1773	98.03%	54.17%	54.17%
	D-Louvain	2416	98.55%	53.20%	53.20%
	Ours	10605	99.36%	96.58%	96.58%

Note: “Events” refers to the number of alerts grouped into security events. “Risk Events (S)” denotes the percentage of pure events that contain only one risk level. “Risk Alerts (S)” and “Risk Alerts (A)” represent the proportions of alerts correctly grouped by the same risk level and by adjacent risk levels, respectively. “D-Louvain” refers to the Louvain algorithm applied in a nested manner twice.

Overall, DEEPDRAC consistently delivers the best performance among all the evaluated methods, with particularly large margins over the second-best approaches on the EIPGC and CIC-IDS2017 datasets. In fact, on the CIC-IDS2017 dataset, the time intervals between different categories of attacks are relatively long, and there are only two risk levels: false positive and high risk (corresponding to attacks). Therefore, security events composed of alerts of a single risk level are equivalent to those composed of alerts with the same triggering cause, and DEEPDRAC achieves 96.58% accurate reconstruction on them. The primary reconstruction challenges for the CIC-IDS2017 dataset arise from botnet events. Among all 59 security events involving two risk levels, 41 events (accounting for 69.49%) are composed of botnet alerts and false positive alerts. The potential impact of these mixed events is further discussed in Section V-B.

Answer to RQ3: DEEPDRAC can accurately reconstruct security events from raw alerts, achieving over 95% recovery accuracy for single-risk security events on all datasets.

E. From Pattern to Disposition: A DeepDRAC Case Study

We present a representative case from the EIPGC dataset to illustrate how DEEPDRAC provides interpretable descriptions for security events and enables automated alert disposition through pattern-based analysis.

First, DEEPDRAC categorizes security events reconstructed from raw alerts into basic patterns, each of which provides an overview of the event. As illustrated in Fig. 8, the basic pattern in this case indicates that multiple unknown IP addresses attempted XSS and injection attacks targeting an internal NAT server. Second, within this basic pattern, DEEPDRAC

further clusters events into three clusters (pattern clusters) using GNN embeddings, which capture detailed information such as the distribution (distri.) of attacks and the number of compromised nodes, making cluster boundaries more distinct. By sampling eight security events from each cluster, SOC operators determined that pattern clusters #1 and #2 are triggered by internal personnel accessing websites potentially containing XSS vulnerabilities via the NAT server, which are non-malicious and thus classified as low-risk (monitor but no action required). In contrast, pattern cluster #3 involves attackers using distributed IP addresses for frequent XML injection attempts, indicating clear malicious intent, prompting IP blocking. The triggering causes and disposal strategies are stored in DEEPDRAC’s pattern knowledge base. Finally, when new security events match pattern clusters #1 and #3, DEEPDRAC immediately recommends corresponding risk levels and disposal strategies for operators’ reference. If operators trust DEEPDRAC’s strategies for these clusters, DEEPDRAC can automatically deploy the response actions.

V. DEFENSE AND DISCUSSION

A. Time Efficiency Analysis

To evaluate the practical feasibility of DEEPDRAC in real-world scenarios, this section presents an analysis of its runtime performance and scalability when dealing with alerts of different scales. We use the CIC-IDS2017 dataset, which contains a diverse range of attack categories, as the source of alert data. To assess time consumption, we evaluate both the model inference and training stages across varying alert scales, with $N_{alert} = 1 \sim 500K$. The results show that during the model training stage, even when trained with 500K alerts, the GNN can complete training on a single NVIDIA GeForce RTX 3090 GPU in 27.8 minutes (compared to 35 minutes for DeepCASE). This is because that the GNN model is designed as a lightweight model with only 71,719 parameters. During the model inference stage, DEEPDRAC achieves efficient processing of large-scale alerts while maintaining near-linear time complexity. Additionally, compared to the standard Louvain algorithm, it exhibits better scalability under high alert volumes. Relevant experimental results and theoretical analyses are provided in Appendix C of the Supplementary material.

B. Detection Performance Across Attack Categories

This section discusses the attack-specific detection performance of DEEPDRAC to provide deeper insights into its strengths and limitations. We evaluate DEEPDRAC across all attack scenarios in the CIC-IDS2017 dataset, focusing on the recall and false-positive rate (FPR) for each attack category. The FPR refers to the proportion of samples that do not belong to a given attack category but are incorrectly predicted as that category.

Table VI in the Supplementary material presents the performance of DEEPDRAC across different attack scenarios. Since Snort3 detected only a very limited number of SQL injection alerts (19 out of 685,560), no recommendation could be matched for this category. For the remaining 12 attack categories, recommendations were successfully generated during

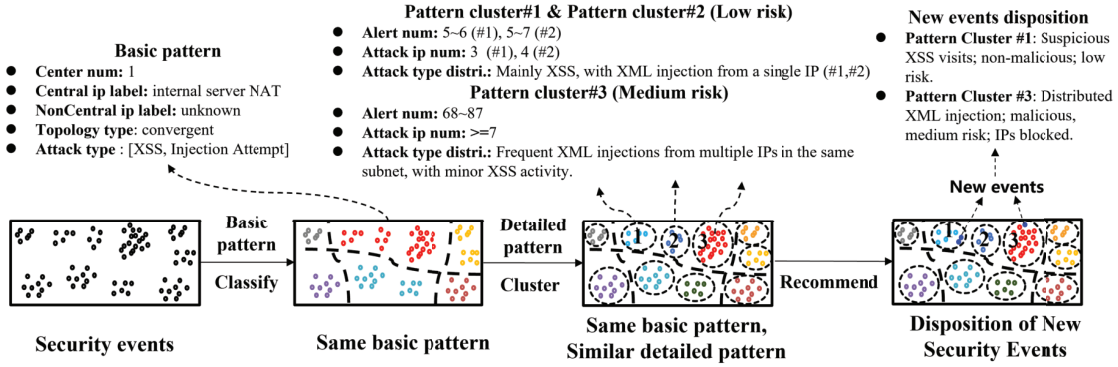


Fig. 8. A case study in DEEPDRAC: Pattern-based analysis and automated disposition in DEEPDRAC. An example from the EIPGC dataset illustrates how DEEPDRAC provides summary descriptions for security events through basic patterns, clusters them into three fine-grained pattern clusters using GNN embeddings, and recommends appropriate risk levels and disposal strategies.

testing. The recommendation results show that DEEPDRAC performs excellently in five attack categories: PortScan, SSH-Patator, Web-Brute Force, DoS Slowloris, and FTP-Patator, achieving a 99% recall rate while maintaining an FPR below 0.1%. For the remaining seven attack categories with relatively lower performance, we analyze the sources of false positives and the types of missed predictions, as detailed in Table VII in the Supplementary material. The analysis leads to the following conclusions:

- (i) DEEPDRAC confuses four categories of DoS attacks (DDoS, DoS Hulk, DoS Slowhttptest, and DoS GoldenEye) but can distinguish DoS from other attack scenarios. Four DoS attack categories serve as the primary sources of false positives, and the categories of missed predictions are the same. Owing to attack characteristics similar to DoS (a large number of requests in a short time), a small number of PortScan events are also confused with DoS attacks.
- (ii) Limited by the extremely scarce training data for the Web-XSS and infiltration attack categories, DEEPDRAC has very limited detection capabilities for them.
- (iii) DEEPDRAC performs the worst on botnet (Bot) attacks, failing to distinguish them from BENIGN alerts. Further analysis reveals that DEEPDRAC reconstructs 56 security events containing Bot alerts, among which 41 are mixed with BENIGN alerts. This causes these security events to exhibit characteristics of both BENIGN and botnet alerts, making it difficult for DEEPDRAC to differentiate between them.

C. Robustness and Stability of the Model

In the historical alerts of the SOC, there may be mislabeled data. This section analyzes the impact of mislabeled data on DEEPDRAC. We simulate the process of operators labeling security events via DEEPDRAC by introducing controlled label noise into the CIC-IDS2017 training set. Specifically, we randomly select between 1% and 100% of the data for corruption, employing the following strategy: 1) If a BENIGN security event is selected, it is randomly relabeled as an arbitrary attack category; 2) if an attack security event is selected, it has a 50% chance of being mislabeled as another

attack category and a 50% chance of being mislabeled as BENIGN. We then retrain the best performing DEEPDRAC model from Section IV-B using the corrupted training set and evaluate its performance in terms of overall alert reduction, recommendation accuracy, and recommendation recall. Fig. 11 in the Supplementary material presents the performance of DEEPDRAC under varying proportions of mislabeled data:

- (i) Recommendation Accuracy: When the mislabeled data size is $\leq 10\%$, fine-tuning still has a positive effect, resulting in higher accuracy than the model without fine-tuning. When the mislabeled data size exceeds 10%, fine-tuning begins to negatively impact accuracy. However, even with 100% mislabeled data, the pretrained knowledge remains effective: GNN embeddings still contribute positively, and the accuracy remains higher than that of the non-GNN model.
- (ii) Recommendation Recall: The recall is barely affected by the mislabeled data size and remains consistently above 99%.
- (iii) Although the ability of DEEPDRAC to reduce the alert analysis workload decreases as the amount of mislabeled data increases, DEEPDRAC still reduces the analysis workload by 97% even when the mislabeled data size is 100%. This capability comes mainly from reconstructing raw alerts into security events, which greatly compresses the analysis scale. In particular, the event reconstruction stage is entirely based on unlabeled data and is decoupled from GNN training.

Overall, even in the presence of mislabeled data, DEEPDRAC still performs effectively. This robustness benefits from the combined effect of multiple DEEPDRAC modules, including unsupervised security event reconstruction, the GNN pretraining strategy, and hierarchical clustering of security events (which first requires matching basic patterns before clustering based on GNN embeddings).

VI. CONCLUSION

In this work, we develop a novel alert validation method, DEEPDRAC, which quickly disposes of alerts through event reconstruction and pattern recommendation. We confirm the efficiency of alert disposal via event reconstruction and pattern recommendation while preserving interpretability well.

Extensive experimental results demonstrate that DEEPDRAC can reduce the workload of alert analysis more effectively than the two baseline methods can. In addition, the fact that DEEPDRAC does not rely on expert knowledge or extensive labeled training data highlights its potential as a useful tool for real-world security operations centers. In the future, we plan to enhance the recognition of rare categories by leveraging synthetic sample generation techniques and refining the enhanced Louvain algorithm to better handle complex attack scenarios such as Botnets.

REFERENCES

- [1] F. B. Kokulu et al., "Matched and mismatched SOC's: A qualitative study on security operations center issues," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1955–1970.
- [2] W. U. Hassan et al., "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–15.
- [3] K. Bhukar et al., "Dynamic- X-Y: A tool for learning dynamic alert suppression policies in AIOps," in *Proc. 16th Int. Conf. Commun. Syst. Netw. (COMSNETS)*, Jan. 2024, pp. 291–293.
- [4] T. Ahmed, A. Shah, M. Kolla, and R. Yellasi, "Reduction of alert fatigue using extended isolation forest," in *Proc. Int. Conf. Forensics, Anal., Big Data, Secur. (FABS)*, Dec. 2021, pp. 1–5.
- [5] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2783–2800.
- [6] X. Wang, X. Yang, X. Liang, X. Zhang, W. Zhang, and X. Gong, "Combating alert fatigue with AlertPro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection," *Comput. Secur.*, vol. 137, Feb. 2024, Art. no. 103583.
- [7] CISCO.(2019). *Anticipating the Unknowns—chief Information Security Officer (CISO) Benchmark Study*. [Online]. Available: <https://ebooks.cisco.com/story/anticipating-unknowns/>
- [8] G. Ho, A. Sharma, M. Javed, V. Paxson, and D. Wagner, "Detecting credential spearphishing in enterprise settings," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 469–485.
- [9] T. E. T. Djaidja, B. Brik, S. M. Senouci, A. Boualouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and RNNs," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7783–7793, 2024.
- [10] J. He, C. Tang, W. Li, T. Li, L. Chen, and X. Lan, "BR-HIDF: An anti-sparsity and effective host intrusion detection framework based on multi-granularity feature extraction," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 485–499, 2024.
- [11] C. Zha et al., "A-NIDS: Adaptive network intrusion detection system based on clustering and stacked CTGAN," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3204–3219, 2025.
- [12] T. V. Ede et al., "DEEPCASE: Semi-supervised contextual analysis of security events," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 522–539.
- [13] N. Hubballi and V. Suryanarayanan, "False alarm minimization techniques in signature-based intrusion detection systems: A survey," *Comput. Commun.*, vol. 49, pp. 1–17, Aug. 2014.
- [14] A. A. Ramaki, A. Rasoolzadegan, and A. G. Bafghi, "A systematic mapping study on intrusion alert analysis in intrusion detection systems," *ACM Comput. Surveys*, vol. 51, no. 3, pp. 1–41, May 2019.
- [15] M. Husák, M. Cermák, M. Laštovická, and J. Vykopal, "Exchanging security events: Which and how many alerts can we aggregate?," in *Proc. IFIP/IEEE Symp. Integr. Netw. Service Manage. (IM)*, May 2017, pp. 604–607.
- [16] X. Luo et al., "BiCAM: A bidirectional contextualized attentive model for analyzing the correlation of heterogeneous security events," *IEEE Trans. Rel.*, vol. 74, no. 2, pp. 2640–2654, Jun. 2025.
- [17] Y. Liu, X. Shu, Y. Sun, J. Jang, and P. Mittal, "RAPID: Real-time alert investigation with context-aware prioritization for efficient threat discovery," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 827–840.
- [18] J. Liu et al., "Context2 Vector: Accelerating security event triage via context representation learning," *Inf. Softw. Technol.*, vol. 146, Jun. 2022, Art. no. 106856.
- [19] A. AlEroud and G. Karabatis, "Beyond data: Contextual information fusion for cyber security analytics," in *Proc. 31st Annu. ACM Symp. Appl. Comput.*, Apr. 2016, pp. 73–79.
- [20] Q. Cheng, C. Wu, and S. Zhou, "Discovering attack scenarios via intrusion alert correlation using graph convolutional networks," *IEEE Commun. Lett.*, vol. 25, no. 5, pp. 1564–1567, May 2021.
- [21] Y. Liu et al., "A context-aware clustering approach for assisting operators in classifying security alerts," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 153–171, Jan. 2025.
- [22] M. E. Aminanto, L. Zhu, T. Ban, R. Isawa, T. Takahashi, and D. Inoue, "Combating threat-alert fatigue with online anomaly detection using isolation forest," in *Proc. 26th Int. Conf. Neural Inf. Process.*, Sydney, NSW, Australia. Cham, Switzerland: Springer, Jan. 2019, pp. 756–765.
- [23] J. Mink et al., "Everybody's got ML, tell me what else you have: Practitioners' perception of ML-based security tools and explanations," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2068–2085.
- [24] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, nos. 3–5, pp. 75–174, Feb. 2010.
- [25] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 76, no. 3, Sep. 2007, Art. no. 036106.
- [26] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, Jun. 2002.
- [27] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 69, no. 2, Feb. 2004, Art. no. 026113.
- [28] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mechanics: Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.
- [29] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," *SIGMOD Rec.*, vol. 29, no. 2, pp. 1–12, May 2000.
- [30] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon, "Network motifs: Simple building blocks of complex networks," *Science*, vol. 298, no. 5594, pp. 824–827, 2002.
- [31] S. Wernicke and F. Rasche, "FANMOD: A tool for fast network motif detection," *Bioinformatics*, vol. 22, no. 9, pp. 1152–1153, May 2006.
- [32] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learning*, vol. 70, 2017, pp. 1263–1272.
- [33] V. P. Dwivedi, A. T. Luu, L. Thomas, Y. Bengio, and X. Bresson, "Graph neural networks with learnable structural and positional representations," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2021, pp. 1–15.
- [34] L. Rampásek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 14501–14515.
- [35] W. Hu et al., "Strategies for pre-training graph neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2019, pp. 1–18.
- [36] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [37] P. Velić ković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2017, pp. 1–12.
- [38] Z. Wang, J. Chen, and H. Chen, "EGAT: Edge-featured graph attention network," in *Proc. Int. Conf. Artif. Neural Netw.*, Bratislava, Slovakia. Cham, Switzerland: Springer, Sep. 2021, pp. 253–264.
- [39] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated graph sequence neural networks," in *Proc. ICLR*, Jan. 2015, pp. 1–20.
- [40] W. Jin et al., "Self-supervised learning on graphs: Deep insights and new direction," 2020, *arXiv:2006.10141*.
- [41] L. Wu, H. Lin, C. Tan, Z. Gao, and S. Z. Li, "Self-supervised learning on graphs: Contrastive, generative, or predictive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4216–4235, Apr. 2023.
- [42] F. Costa and K. D. Grave, "Fast neighborhood subgraph pairwise distance kernel," in *Proc. 27th Int. Conf. Int. Conf. Mach. Learn.*, Jun. 2010, pp. 255–262.
- [43] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3835–3845.
- [44] (1999). *DARPA Intrusion Detection Evaluation Dataset*. [Online]. Available: <https://archive.ll.mit.edu/ideval/data/1999data.html>
- [45] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, vol. 1, 2018, pp. 108–116.